CSE546: Applied Cryptography
Assignment 3

Name: Shreyansh Seth
Roll No.: 2022484
Email: `shreyansh22484@iiitd.ac.in`

Latex Document (Read Only)

I have attempted **Question 1, Question 2, and Question 5**. For the programming parts, I ran my Python scripts and I attach screenshots of the execution outputs at the exact places where they are referenced: `1a.png` (Q1 encryption + verification), `1b.png` (Q1 nonce-reuse demo), and `5a.png`/`5b.png` (Q5 code proof + output proof).

Question 1: AES-GCM (Authenticated Encryption)

1.1 AES-GCM encryption, tag generation, and verification

I implemented AES-GCM for a single message using AES-128 in CTR mode for confidentiality and GHASH over $\mathbb{F}_{2^{128}}$ for integrity/authenticity. I followed the standard 96-bit nonce construction:

$$J_0 = N \parallel 0^{31} \parallel 1,$$

and generated the CTR keystream using $\mathrm{inc32}(J_0), \mathrm{inc32}^2(J_0), \ldots$.

Given inputs:

$$P = \texttt{00112233445566778899aabbccddeeff},$$
$$K = \texttt{000102030405060708090a0b0c0d0e0f},$$
$$N = \texttt{aabbccddeeff001122334455}.$$

I used empty associated data ($A = \epsilon$).

Computed outputs (from my run):

$$C = \texttt{62db4e89c0ca387a1933c394808e9158}, \qquad T = \texttt{b4a00ef1da31110bbe4448ddc89c8b92}.$$

I decrypted back to the original plaintext and validated the recomputed tag; verification succeeded (see Figure 1).

1.2 Authenticated encryption properties

- **What does AES-GCM provide that AES-CTR alone does not?**
  AES-CTR provides confidentiality only, but it is malleable: if an attacker flips a ciphertext bit, the corresponding plaintext bit flips after decryption with no detection. AES-GCM is an AEAD scheme, so it provides confidentiality and integrity/authenticity using an authentication tag. Any modification of the ciphertext (or associated data) is detected during tag verification. In practical terms, CTR gives encryption, while GCM gives encryption with tamper detection under nonce uniqueness.

- **Role of GHASH in integrity/authenticity.**
  GHASH is a polynomial-based hash over $\mathbb{F}_{2^{128}}$ computed using a secret subkey $H = E_K(0^{128})$. It hashes the associated data and ciphertext into a 128-bit value, and the final tag is:

$$T = E_K(J_0) \oplus \mathrm{GHASH}_H(A, C).$$

```
=== AES-128 GCM ===
Key (K)            : 000102030405060708090a0b0c0d0e0f
96-bit Nonce (N)   : aabbccddeeff001122334455
Plaintext (P)      : 00112233445566778899aabbccddeeff
Ciphertext (C)     : 62db4e89c0ca387a1933c394808e9158
Auth Tag (T)       : b4a00ef1da31110bbe4448ddc89c8b92

=== Decryption & Tag Verification ===
Decrypted Plaintext: 00112233445566778899aabbccddeeff
Recomputed Tag     : b4a00ef1da31110bbe4448ddc89c8b92
Tag Verified?      : True
```

Figure 1: Q1(1.1): My AES-GCM encryption output and successful decryption + tag verification.

Without knowing H (and the mask $E_K(J_0)$), an attacker cannot feasibly modify A or C while still producing a valid tag, except with negligible probability.

- **Why authenticate both ciphertext and associated data?**
  Associated data (AAD) is often header/context information that must remain in plaintext but still must be protected from tampering. Authenticating both A and C ensures the receiver accepts the message only if the ciphertext and its context/metadata match what I intended to send. Otherwise, an attacker could alter headers (identities, flags, protocol fields, etc.) while leaving C unchanged and still pass decryption if AAD were not authenticated.

1.3 Nonce misuse and security failures

- **What happens if the same nonce is reused with the same key?**
  Reusing the nonce with the same key repeats J_0 and repeats the CTR keystream, which breaks confidentiality. It also creates exploitable algebraic relations between authentication tags because the same mask $E_K(J_0)$ is reused.

- **Plaintext recovery from CTR keystream reuse.**
  CTR encryption is $C = P \oplus KS$. If the same key+nonce are reused, the keystream KS repeats:

  $$C_1 = P_1 \oplus KS, \qquad C_2 = P_2 \oplus KS.$$

  XORing cancels the keystream:

  $$C_1 \oplus C_2 = (P_1 \oplus KS) \oplus (P_2 \oplus KS) = P_1 \oplus P_2.$$

  So an attacker learns $P_1 \oplus P_2$. If P_1 is known/guessable, then $P_2 = (C_1 \oplus C_2) \oplus P_1$.

- **Why authenticity can break completely (GHASH linearity).**
  GCM uses:

  $$T = E_K(J_0) \oplus \mathrm{GHASH}_H(A, C).$$

  With nonce reuse, $E_K(J_0)$ repeats, so:

  $$T_1 \oplus T_2 = \mathrm{GHASH}_H(A_1, C_1) \oplus \mathrm{GHASH}_H(A_2, C_2).$$

  GHASH is linear over XOR, so this gives strong algebraic relations that can be exploited (especially in structured/special cases) to forge valid tags under the reused nonce. In my demo run I showed keystream reuse and also constructed a valid forgery under the reused nonce (Figure 2).

```
=== Nonce Reuse Attack Demo (Same Key + Same Nonce) ===
K: 000102030405060708090a0b0c0d0e0f
N: aabbccddeeff001122334455

Message 1
P1: 00112233445566778899aabbccddeeff
C1: 62db4e89c0ca387a1933c394808e9158
T1: b4a00ef1da31110bbe4448ddc89c8b92

Message 2
P2: 10213243546576879 8a9babbdcddedef
C2: 72eb5ef9d0fa288a0903d394908e9248
T2: 773aae7829ff18c03f77562628665e12

=== Confidentiality Break (CTR Keystream Reuse) ===
C1 = P1 XOR KS,  C2 = P2 XOR KS
So: C1 XOR C2 = P1 XOR P2
C1 XOR C2: 10301070103010f01030100010000310
P1 XOR P2: 10301070103010f01030100010000310
Equal?    : True

If attacker knows P1, they recover P2:
P2 = (C1 XOR C2) XOR P1
Recovered P2: 10213243546576879 8a9babbdcddedef
Matches P2? : True

Recover keystream if one plaintext is known:
KS = C1 XOR P1
Recovered KS: 62ca6cba849f5e0d91aa692f4c537fa7

=== Authenticity Break (Tag Relation + Forgery) ===
T = E(K,J0) XOR GHASH_H(A, C)
Same nonce => same J0 => same E(K,J0)
So: T1 XOR T2 = GHASH_H(A,C1) XOR GHASH_H(A,C2)

1-block, empty AAD case:
T1 XOR T2 = (C1 XOR C2) * H^2
So attacker gets H^2, then H = sqrt(H^2), then can compute E(K,J0)
Recovered H (attacker): c6a13b37878f5b826f4f8162a1c8d879
Real H (from key)     : c6a13b37878f5b826f4f8162a1c8d879
H correct?            : True

Forge a new valid (C3, T3) under reused nonce:
Choose P3, set C3 = P3 XOR KS, set T3 = E(K,J0) XOR GHASH_H(A,C3)
P3 (chosen) : ffffffffffffffff0000000000000000
C3 (forged) : 9d3593457b60a1f291aa692f4c537fa7
T3 (forged) : a0603b184a5b9ffd87b1c73e112f6aac
Tag Verified? : True
Decrypt Out   : ffffffffffffffff0000000000000000
Matches P3?   : True
```

Figure 2: Q1(1.3): My nonce-reuse demo showing confidentiality break and a successful forgery when nonce is reused.

1.4 Comparison with other AEAD modes

(1) AES-GCM vs AES-CBC + HMAC.

AES-CBC + HMAC (when done correctly as Encrypt-then-MAC) provides confidentiality and integrity, but it typically requires two passes (CBC encryption plus a separate MAC computation). CBC encryption is not parallelizable across blocks, while GCM uses CTR mode (parallelizable) and GHASH (also parallelizable), making GCM generally faster on modern platforms. CBC-based schemes are also historically prone to padding-oracle pitfalls if the MAC check and error handling are implemented incorrectly. GCM is a clean standardized AEAD construction, but it requires strict nonce uniqueness.

(2) AES-GCM vs ChaCha20-Poly1305 (software-only).
On machines without AES hardware acceleration, ChaCha20-Poly1305 is often faster because ChaCha20 uses ARX operations (add/rotate/xor) designed for high-speed software. Both are AEAD schemes and both require nonce uniqueness; reuse repeats a keystream and can lead to confidentiality loss, and can also enable integrity failures. Poly1305 is efficient and commonly used on mobile and embedded devices. On platforms with AES-NI and carry-less multiply, AES-GCM often becomes the faster option.

(3) One limitation/trade-off of AES-GCM.
The main limitation is catastrophic failure on nonce reuse: repeating a nonce under the same key can leak relations between plaintexts and can enable tag forgeries. GHASH also relies on finite-field multiplication, which can be slower in pure software on systems lacking carry-less multiplication instructions. In practice, secure deployment requires careful nonce management (counters or uniqueness guarantees), which can be operationally tricky in distributed systems.

Question 2: ECB, CBC, and CTR (PRP-based analysis)

Let $E_K(\cdot)$ be an ideal PRP on 128-bit blocks and $M = (M_1, M_2, M_3, M_4)$.

(a) ECB mode

In ECB, each block is encrypted independently:

$$C_i = E_K(M_i) \quad \text{for } i = 1, 2, 3, 4.$$

So:

$$C_1 = E_K(M_1),\ C_2 = E_K(M_2),\ C_3 = E_K(M_3),\ C_4 = E_K(M_4).$$

Given $M_1 = M_3$, determinism under fixed K implies $C_1 = C_3$. Since $M_2 \neq M_4$ and E_K is a permutation, $C_2 \neq C_4$. Therefore, an adversary can detect repeated blocks in the plaintext from repeated ciphertext blocks, leaking the plaintext's repetition/structure.

(b) CBC mode with random IV

CBC encryption with uniformly random IV:

$$C_1 = E_K(M_1 \oplus IV), \qquad C_i = E_K(M_i \oplus C_{i-1}) \text{ for } i = 2, 3, 4.$$

So:

$$C_1 = E_K(M_1 \oplus IV),\ C_2 = E_K(M_2 \oplus C_1),\ C_3 = E_K(M_3 \oplus C_2),\ C_4 = E_K(M_4 \oplus C_3).$$

It is not guaranteed that $C_1 = C_3$. Even if $M_1 = M_3$, the chaining makes the inputs to $E_K(\cdot)$ different because they depend on different previous ciphertext blocks. If $C_1 = C_3$, then by injectivity of E_K:

$$M_1 \oplus IV = M_3 \oplus C_2.$$

Using $M_1 = M_3$ gives $IV = C_2$, which happens with negligible probability (about 2^{-128}) for random IV and pseudorandom-looking C_2. Therefore $C_1 = C_3$ is not guaranteed in CBC even when $M_1 = M_3$.

(c) CTR mode with AES-128

(a) Why encryption and decryption are the same operation

CTR generates keystream blocks:

$$KS_i = E_K(\text{Nonce} \parallel (\text{ctr} + i)),$$

and encrypts as $C_i = P_i \oplus KS_i$. Decryption uses the same XOR: $P_i = C_i \oplus KS_i$, because XOR is its own inverse.

(b) Reusing the same nonce–counter pair

CTR must use a unique nonce–counter sequence per key because it is a stream-cipher construction; reusing it repeats the keystream. If the same nonce–counter sequence is reused for two different messages, then the same keystream is reused:

$$C_i = P_i \oplus KS_i, \qquad C'_i = P'_i \oplus KS_i.$$

XORing cancels the keystream:

$$C_i \oplus C'_i = P_i \oplus P'_i.$$

So an attacker can compute the XOR of the plaintexts directly as $P_i \oplus P'_i = C_i \oplus C'_i$.

Question 5: Ascon (S-box analysis and verification)

1(a) S-box and truth table

I implemented `ascon_sbox(x)` exactly from the given Boolean equations (with $x[0]$ as LSB and $x[4]$ as MSB). I also include my code proof screenshot for the S-box implementation (Figure 3). Using the implementation, I computed the truth table:

$$[S(0), S(1), \ldots, S(31)] = [0, 3, 6, 5, 12, 31, 10, 25, 24, 31, 31, 24, 20, 3, 19, 4, \\ 17, 18, 31, 28, 31, 12, 17, 2, 9, 14, 6, 1, 7, 16, 8, 31].$$

```python
def ascon_sbox(x: int) -> int:
    if x < 0 or x >= 32:
        raise ValueError("x must be a 5-bit integer (0..31)")
    x0 = (x >> 0) & 1
    x1 = (x >> 1) & 1
    x2 = (x >> 2) & 1
    x3 = (x >> 3) & 1
    x4 = (x >> 4) & 1

    y0 = x4 ^ x0 ^ (x3 & x1)
    y1 = x0 ^ x1 ^ (x4 & x2)
    y2 = x1 ^ x2 ^ (x0 & x3)
    y3 = x2 ^ x3 ^ (x1 & x4)
    y4 = x3 ^ x4 ^ (x2 & x0)

    return (y0 << 0) | (y1 << 1) | (y2 << 2) | (y3 << 3) | (y4 << 4)
```

Figure 3: Q5 code proof: My `ascon_sbox(x)` implementation used to compute the truth table.

1(b) S-box properties

- **Nonlinearity:** I computed the nonlinearity as 8.
- **Involutory?** No. Counterexample: $x = 1$, $S(1) = 3$, but $S(S(1)) = S(3) = 5 \neq 1$.

2) Simplified round outputs and official library verification

Using the specified toy round:

$$\begin{aligned} &\text{(i) } state \leftarrow S(state), \\ &\text{(ii) } state \leftarrow state \oplus ((state \ll 1)\&0x1F) \oplus ((state \gg 4)\&0x1F), \\ &\text{(iii) } state \leftarrow state \oplus rc. \end{aligned}$$

I obtained:

$$\texttt{simplified_ascon_round(0b00101, 0b10000)} = 16 \ (0b10000),$$

$$\texttt{simplified_ascon_round(0b11111, 0b00001)} = 1 \ (0b00001).$$

I also verified encryption using the official `ascon` Python library with:

- `plaintext=b'CRYPTOGRAPHYFUN!!'`
- `key=b'0123456789ABCDEF'`

- `nonce=b'ABCDEFGHIJKLMNOP'`
- `associateddata=None`, `variant="Ascon-128"`

The first 4 bytes of ciphertext (8 hex characters) I obtained are:

$$\boxed{\texttt{7037a3a2}}.$$

3) Mode choice for using only the permutation

If I want to use only the 320-bit permutation as a building block for encryption (without a traditional key schedule), I would choose **CTR mode**. CTR fits naturally because I can generate a pseudorandom keystream from the permutation and XOR it with the plaintext, and it is parallelizable. ECB leaks patterns and CBC requires block-cipher-style chaining and IV handling; CTR avoids pattern leakage when nonces/counters are unique.

```
=== Part 1(a): ascon_sbox(x) and Truth Table ===
TruthTable = [0, 3, 6, 5, 12, 31, 10, 25, 24, 31, 31, 24, 20, 3, 19, 4, 17, 18, 31, 28, 31, 12, 17, 2, 9, 14, 6, 1, 7, 16, 8, 31]

=== Part 1(b): Properties ===
Nonlinearity = 8
WorstMaskA = 00001 decimal 1
WorstWalshW = 10001 decimal 17
MaxAbsWalsh = 16
Involutory? = False
Counterexample: x = 1 S(x) = 3 S(S(x)) = 5

=== Part 2: simplified_ascon_round(state, round_constant) ===
Input state=0b00101, rc=0b10000 -> output = 16 binary 10000
Input state=0b11111, rc=0b00001 -> output = 1 binary 00001

=== Part 2: Official Ascon Library Encryption Check ===
CiphertextFirst4Bytes (8 hex chars): 7037a3a2

=== Part 4(a): DDT Entry for Δx=1 and Δy=1 ===
Count = 0
```

Figure 4: Q5 output proof: Truth table, S-box properties, toy-round outputs, official library ciphertext prefix, and DDT count from my run.

4(a) DDT entry for $\Delta x = 1$ and $\Delta y = 1$

Using the truth table, I counted the number of x such that:

$$S(x) \oplus S(x \oplus 1) = 1.$$

My computed count is:

$$\boxed{0}.$$

4(b) Differential resistance and wide-trail argument

Because the DDT entry for $(\Delta x, \Delta y) = (1, 1)$ is 0, that specific differential transition is impossible (probability 0) for this S-box. More generally, the AND terms in the S-box make it nonlinear, which reduces the probability that an input difference leads to a specific output difference with high probability. In the full Ascon permutation, the diffusion layer spreads differences across many bits/words so that the number of active S-boxes increases quickly across rounds. By the wide-trail strategy, the probability of a multi-round differential trail is bounded by the product of per-S-box probabilities (roughly $(p_{\max})^{\#\text{active S-boxes}}$), and diffusion forces the number of active S-boxes to grow, pushing differential probabilities down exponentially over rounds.